

Tecnicatura Universitaria en Programación - UTN

TRABAJO PRÁCTICO INTEGRADOR

Gestión de Datos de Países

Alumnos: Facundo Bailo - Comisión 2 | Santino Fiorentini - Comisión 4

Grupo: 145

Materia: Programación 1

Coordinador: Alberto Cortez

Docentes tutores: Oscar Londero – Comisión 2 | Ana Mutti – Comisión 4

Fecha de entrega: 11 de noviembre de 2025

Video explicativo: <https://www.youtube.com/watch?v=Cmn1sexm3MI>

Repositorio: <https://github.com/santino-fiorentini/TPI-Programacion1>

Marco teórico:

- Listas:

En Python, una lista es una estructura de datos que permite almacenar múltiples elementos en una sola variable. Son mutables, lo que significa que pueden modificarse durante la ejecución del programa (agregar, eliminar o cambiar elementos).

En el proyecto se utilizan para guardar el conjunto de países cargados desde el archivo CSV.

Ejemplo:

```
países = cargar_paises(UBICACION_DATA)
```

Cada elemento de esa lista es un diccionario con la información de un país.

- Diccionarios:

Un diccionario almacena pares clave:valor. En este trabajo se usa para representar los datos de cada país.

Ejemplo:

```
pais = {'nombre': nombre, 'poblacion': poblacion_parseada, 'superficie':  
superficie_parseada, 'continente': continente}
```

Los diccionarios son ideales para manejar información estructurada, ya que cada campo tiene un nombre descriptivo y se puede modificar fácilmente.

- Funciones:

Una función es un bloque de código que realiza una tarea específica y puede reutilizarse varias veces dentro del programa. Este enfoque se conoce como modularización.

Ejemplo:

```
def validar_texto(texto):  
    if not texto or texto.strip() == '':  
        return False  
    return True
```

Cada función tiene una única responsabilidad, como: cargar_paises(); lee el archivo CSV, agregar_pais(); permite agregar un país nuevo, filtrar_paises(); aplica filtros de búsqueda, mostrar_estadisticas(); muestra los cálculos estadísticos.

El uso de funciones mejora la legibilidad, facilita el mantenimiento y evita repetir código.

- Condicionales:

Los condicionales permiten que el programa tome decisiones. En Python se implementan con if, elif y else.

Ejemplo:

```
if not paises:  
    print("No hay paises para listar.")  
    return
```

Esto verifica si la lista paises está vacía.

También se usa la estructura match/case para controlar las opciones del menú:

```
match opc:  
    case 1:  
        agregar_pais(paises=paises, dataset=UBICACION_DATA)  
    case 2:  
        actualizar_pais(paises=paises, dataset=UBICACION_DATA)  
    case 3:  
        buscar_pais(paises)  
    case 4:  
        filtrar_paises(paises)  
    case 5:  
        ordenar_paises(paises)  
    case 6:  
        mostrar_estadisticas(paises)  
    case 7:  
        print("Programa finalizado.")  
        break  
    case _:  
        print("Opción inválida.")
```

- Ordenamientos:

Ordenar datos significa organizarlos según un criterio, por ejemplo, alfabéticamente o por tamaño numérico. En Python se utiliza la función sorted() junto con una función clave (key).

Ejemplo:

```
def ordenar_por_nombre(paises):  
    paises_ordenados = sorted(paises, key=obtener_nombre)  
    for pais in paises_ordenados:  
        mostrar_pais(pais)  
    return paises_ordenados
```

Esto ordena los países alfabéticamente.

- Estadísticas básicas:

El trabajo incluye funciones que calculan información a partir de los datos cargados.

Ejemplo:

```
def estadistica_promedio_poblacion(países):  
    if not países:  
        print("No hay países para calcular el promedio de población.")  
        return  
    total_poblacion = sum(p['poblacion'] for p in países)  
    promedio_poblacion = total_poblacion / len(países)  
    print(f"El promedio de población de los países es  
{promedio_poblacion:.2f} habitantes.")
```

Estas estadísticas ayudan a analizar los datos y comprobar que los cálculos se realizan correctamente.

- Archivos CSV:

Un archivo CSV (Comma-Separated Values) permite almacenar datos en texto plano separados por comas. Python dispone del módulo csv para leer y escribir este tipo de archivos.

Ejemplo:

```
if os.path.exists(dataset):  
    with open(dataset, 'r', encoding='utf-8') as archivo:  
        reader = csv.DictReader(archivo)  
        for indice, registro in enumerate(reader):  
            pais = validar_y_parsear_registro(registro)  
            if pais:  
                países.append(pais)
```

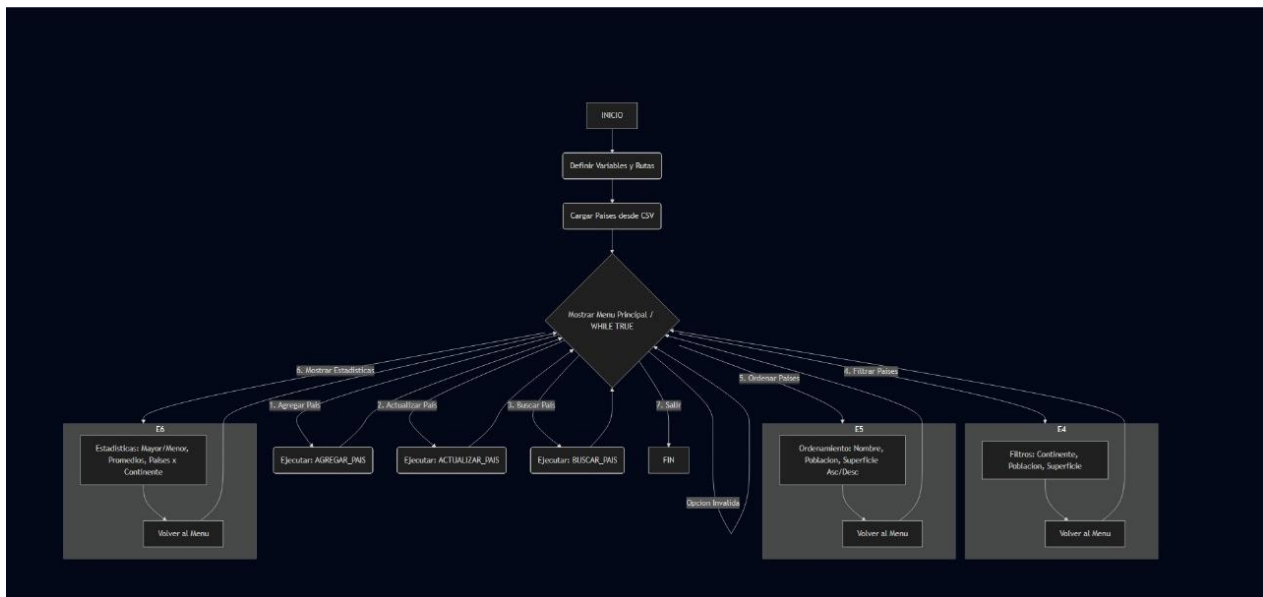
Esto sirve para cargar los datos de un archivo CSV y convertirlos en una lista de diccionarios.

Objetivo del trabajo:

El objetivo del trabajo práctico integrador es desarrollar un programa en Python que permita gestionar países mediante operaciones de carga, actualización, búsqueda, filtrado, ordenamiento y análisis estadístico. El sistema debe leer los datos desde un archivo CSV, procesarlos utilizando estructuras de datos apropiadas (listas y diccionarios) y ofrecer una interfaz de menú interactivo que aplique conceptos fundamentales de programación estructurada.

Diseño del caso práctico:

El diagrama muestra el flujo general del programa Gestor de Países, desde la carga inicial del archivo CSV hasta las distintas operaciones del menú principal.



Metodología utilizada:

El desarrollo del trabajo se basó utilizando funciones específicas para cada tarea con el fin de mantener un código claro, modular y fácil de mantener. Se inició definiendo las variables, rutas y estructuras necesarias para manipular los datos almacenados en un archivo CSV, el cual actúa como base de datos principal. Luego, se implementó la carga de información desde el archivo, junto con validaciones para asegurar la integridad de los registros. El menú principal fue diseñado para ofrecer una interacción ordenada, permitiendo al usuario acceder de forma intuitiva a las distintas funcionalidades del sistema.

Resultados obtenidos:

El programa permite gestionar eficientemente países, ofreciendo operaciones completas como el agregado, búsqueda, actualización, filtrado, ordenamiento y cálculo de estadísticas. Cada opción fue probada para garantizar su correcto funcionamiento y la coherencia de los datos en todo momento. Se logró una interfaz en consola clara y amigable. El sistema demostró ser estable, cumpliendo con las restricciones establecidas y reflejando un manejo adecuado de archivos, estructuras de datos y control de flujo.

Conclusión:

Durante el desarrollo de este trabajo aplicamos de forma integrada los contenidos de Programación I, utilizando listas, diccionarios, funciones, condicionales y manejo de archivos CSV. Logramos construir un sistema funcional que permite gestionar datos reales y generar estadísticas automáticas. El trabajo en equipo permitió dividir tareas, validar resultados y mejorar la claridad del código. Como mejora futura, se podría agregar manejo de excepciones más avanzado y una interfaz gráfica simple para la interacción con el usuario.

Bibliografía:

Material de Cátedra de la materia Programación 1.